Module 3 — Customization

Lesson 3.4

Custom slash commands

Drop a markdown file in .claude/commands/. The filename becomes the command; the body is the prompt.

Why it matters

Custom slash commands turn repeatable prompts into named verbs. Once `/changelog`, `/release-notes`, `/triage`, or `/db-migrate-plan` exists in your repo, anyone on the team can invoke it without re-discovering the right prompt every time.

The mechanics

A custom command is a markdown file in:

- `<project>/.claude/commands/` — project-scoped (committed; shared)
- `~/.claude/commands/` — user-global

Filename → command name. `.claude/commands/changelog.md` becomes `/changelog`.

Minimal example

`.claude/commands/changelog.md`:

```
---
description: Generate a changelog from commits in the given range
---

Generate a changelog from git commits in the range $ARGUMENTS.

Group entries by type: Features, Fixes, Refactors, Docs, Chores.

Format as markdown with a `##` heading per group and a bullet per commit.
Skip "Co-Authored-By" lines. Skip merge commits. Skip commits whose
message starts with "wip" or "fixup!".

Output the changelog as a single markdown block.
```

Invoke: `/changelog v1.2.0..HEAD`

The `$ARGUMENTS` placeholder is replaced with whatever the user types after the command name.

Runnable demo: examples/01-custom-slash-command.

Frontmatter

Supported fields (check current docs for additions):

```
---
description: One-line summary shown in /help
argument-hint: <range>          # shown in command picker
model: claude-haiku-4-5         # override the session model for this command
allowed-tools: Read, Grep, Bash # restrict tool access
---
```

`allowed-tools` is the most underused — it lets a command run in a constrained mode. A `/changelog` command doesn't need `Edit`; restricting tools makes the command safer and faster.

Namespacing

Commands in subdirectories namespace automatically. `.claude/commands/release/notes.md` becomes `/release:notes`. Use this when you have a family of related commands.

Real-world patterns

`/triage <issue-url>`

Fetch a GitHub issue, suggest labels, draft a comment, and (optionally) post.

```
---
description: Triage a GitHub issue
allowed-tools: Bash(gh issue:*), Read, Grep
---

Triage the issue at $ARGUMENTS.

1. Fetch the issue body with `gh issue view`.
2. Search this repo for similar past issues (use Grep on `docs/changelog.md`).
3. Suggest 1–3 labels from this set: bug, feature, docs, question, dup.
4. Draft a 2-sentence reply asking for missing info if any.

Output: labels (comma-separated), then the draft reply in a code block.
Don't post anything — just show the draft.
```

`/release-notes`

Generate user-facing release notes from a tag range, with each entry rewritten for end-users (not engineers).

`/db-migrate-plan`

Take a description of a schema change and produce a phased migration plan: nullable add → backfill → switch reads → switch writes → drop.

`/onboard`

When run, prints the project's onboarding checklist for a new dev: dependencies to install, services to spin up, dashboards to bookmark.

Comparing to MCP-provided commands

MCP servers can also expose commands. The difference:

- **Custom slash commands** are markdown prompts. Authored by you, live in `.claude/commands/`.
- **MCP commands** are exposed by a running MCP server, usually code-backed.

Use a custom slash command when the work is "ask Claude to do X with these instructions." Use MCP when the work needs server-side code (talking to your DB, calling your API).

Try it

1. Create `.claude/commands/standup.md` with a prompt that generates a standup summary from `git log --author="$(git config user.email)" --since=yesterday`.
2. Run `/standup`. Iterate on the prompt until the output is something you'd actually send your team.
3. Add an `allowed-tools` line restricting it to `Bash(git log:*)`.
4. Commit it. Now your team has it too.

Gotchas

- **Filename must be slug-safe.** No spaces, lowercase, hyphens for word separators.
- **`$ARGUMENTS` is literal substitution.** If a user types `$ARGUMENTS` in their args, weird things may happen. Sanitize in the prompt if needed.
- **Commands don't run code — they prompt Claude.** A `/build` command that says "run the build" still requires Claude to invoke Bash. The command is the prompt, not the action.
- **Project commands override user-global commands of the same name.** Useful for project-specific variants.

Further reading

- examples/01-custom-slash-command — runnable
- 5.4 MCP prompts — when to use MCP prompts instead
- Official slash command docs: https://docs.claude.com/en/docs/claude-code/slash-commands

Next

→ 3.5 Statuslines and keybindings